Lesson 2 Worksheet: Tuples — ANSWER KEY

Part 1: Creating Tuples

Description	Code
The origin (row 0, column 0)	`position = (0, 0)`
Row 2, column 3	`position = (2, 3)`
Row 3, column 1	`position = (3, 1)`
Row 0, column 4	`position = (0, 4)`

What characters surround a tuple's values? Parentheses `()`

What separates the values inside a tuple? A comma `,`

Part 2: Indexing Tuples

Given this tuple:

```
position = (2, 3)
```

Expression	Value
`position[0]`	`2`
`position[1]`	`3`

1. **Which index gives you the row?** `0`
2. **Which index gives you the column?** `1`
3. **Does indexing start at 0 or 1?** `0`

Part 3: Predict the Output

Snippet A:

```
start = (0, 0)
print(start)
```

Output: `(0, 0)`

Snippet B:

```
dest = (3, 2)
print("Row:", dest[0])
print("Col:", dest[1])
```

Output line 1: `Row: 3`

Output line 2: `Col: 2`

Snippet C:

```
a = (1, 2)
b = (1, 2)
print(a == b)
```

Output: `True`

Snippet D:

```
a = (1, 2)
b = (2, 1)
print(a == b)
```

Output: `False`

Why are Snippets C and D different?

Order matters in tuples. In Snippet C both tuples have the same values in the same positions, so they are equal. In Snippet D the values are swapped — `(1, 2)` is not the same as `(2, 1)` because index 0 has a different value in each.

Snippet E:

```
point = (3, 1)
row = point[0]
col = point[1]
print("Position is row", row, "column", col)
```

Output: `Position is row 3 column 1`

Part 4: Immutability

Immutable means: It cannot be changed after it is created. Once a tuple is created, you cannot modify, add, or remove its values.

1. **What happens when you run this code?**

```
position = (2, 3)
position[0] = 5
```

Answer: Python raises a `TypeError` because tuples are immutable — you cannot assign a new value to an index of a tuple.

2. **If you want to represent a new position, what do you do instead of modifying the tuple?**

Answer: Create a brand-new tuple with the values you want.

Write the code to create a new tuple for row 5, column 3:

```
new_position = (5, 3)
```

3. **Why is immutability a GOOD thing for coordinates?**

It prevents accidental changes. A coordinate like (2, 3) should always mean row 2, column 3. If it could be changed somewhere else in the code, it could cause hard-to-find bugs. Immutability guarantees the value stays the same once created.

Part 5: Tuple Math — Manhattan Distance

Start	Destination	Row Distance	Col Distance	Manhattan Distance
(0, 0)	(2, 3)	|2 - 0| = 2	|3 - 0| = 3	5
(0, 0)	(3, 0)	|3 - 0| = 3	|0 - 0| = 0	3
(1, 1)	(3, 3)	|3 - 1| = 2	|3 - 1| = 2	4
(2, 3)	(0, 1)	|0 - 2| = 2	|1 - 3| = 2	4
(1, 2)	(1, 2)	|1 - 1| = 0	|2 - 2| = 0	0

Which pair of positions is closest together? (1, 2) to (1, 2) — they are the same point, distance 0.

Which pair is farthest apart? (0, 0) to (2, 3) — Manhattan distance of 5.

Part 6: Code Tracing

```
home = (0, 0)
school = (3, 2)

row_diff = school[0] - home[0]
col_diff = school[1] - home[1]
total = row_diff + col_diff

print("Rows:", row_diff)
print("Cols:", col_diff)
print("Total:", total)
```

Variable	Value
`home[0]`	`0`
`home[1]`	`0`
`school[0]`	`3`
`school[1]`	`2`
`row_diff`	`3`
`col_diff`	`2`
`total`	`5`

Output line 1: `Rows: 3`

Output line 2: `Cols: 2`

Output line 3: `Total: 5`

Part 7: Spot the Error

Error 1:

```
position = (2, 3)
row = position[1]    # Getting the row
```

Problem: Index `[1]` gives the column, not the row. The row is at index `[0]`.

Fix: `row = position[0]`

Error 2:

```
position = [2, 3]    # Storing a coordinate
```

Problem: This creates a list (square brackets `[]`), not a tuple. Lists are mutable, so the coordinate could be accidentally changed.

Fix: `position = (2, 3)` — use parentheses to create a tuple.

Error 3:

```
position = (2, 3)
position[0] = 4      # Move to row 4
```

Problem: Tuples are immutable — you cannot change a value inside a tuple. This raises a `TypeError`.

Fix: Create a new tuple instead: `position = (4, 3)`

Part 8: Tuple vs. Two Variables

1. **If you have 5 different positions, how many variables does Version A need? 10** (2 variables per position: a row and a column for each)

2. **How many variables does Version B need for 5 positions? 5** (1 tuple variable per position)

3. **Which version is easier to pass to a function? Why?**

   Version B (the tuple) is easier to pass to a function because you only pass one argument instead of two. A function call like `move_to(position)` is simpler and less error-prone than `move_to(row, col)`, especially when you have many positions to manage.

Reflection

Explain in your own words: What is a tuple and why is it useful for storing coordinates?

A tuple is an immutable sequence of values grouped together using parentheses and separated by commas. It is useful for storing coordinates because a grid position naturally has two parts (row and column) that belong together. By packing them into a single tuple, you keep them organized as one unit, you can access each part with indexing (`[0]` for row, `[1]` for column), and immutability ensures the coordinate cannot be accidentally changed after it is created.